Ders 3 Çalışma Özeti

Ders 3 Çalışma Özeti

Genel Konular

- Kısa tekrar ve Factory Method'a giriş
  - Geçen derste Abstract Factory kalıbı ve oyun motoru örneğinin üç farklı çözümü (switch-case, kalıtım, kalıp) karşılaştırılmıştı. Bu derste yaratımsal (creational) kalıpların kalanı işlendi: Factory Method, Builder, Prototype ve Singleton.
  - Adında "factory" geçen bir kalıp, sınıf ya da metot nesne oluşturmakla ilgilidir; bu terim tasarım kalıpları dışında da aynı anlamda kullanılır.
- Factory Method: amaç, örnek, sorun (ekrandan, 00:10:00)
  - Amaç: nesne oluşturmak için öyle bir arayüz sunmak ki, oluşturulacak nesnenin sınıfına bu arayüzü sağlayan sınıfın alt sınıfları karar verebilsin.
  - Örnek: farklı belge türleriyle çalışabilecek uygulamalar geliştirmeyi sağlayan bir çerçeve program (framework). Temel sınıflar Belge ve Uygulama; kullanıcı bu iki sınıftan kalıtımla yeni sınıflar türeterek kendi yazılımını hazırlar.
  - Sorun: bir belge oluşturmak gerekince (aç, yeni vb. komutlar) çerçeve program bunu nasıl yapacak? Uygulama, kullanıcının türeteceği yeni belge sınıfları hakkında önceden bilgi sahibi olamaz.
  - Framework "yarım yazılım"dır; programcıların yazacağı uygulamalar sayısız olduğu için öngörülmeyen ihtiyaçlara esnek yanıt verebilmelidir.
- Örnek çözüm: Document ve Application (ekrandan, 00:12:00)

```
Document {Open() abstract, Close() abstract, Save(), Revert()}
   △── MyDocument
Application {CreateDocument() abstract, NewDocument(), OpenDocument()}  ◇── docs ──> Docu
   △── MyApplication {CreateDocument()}  ··> MyDocument
NewDocument():           Document* doc = CreateDocument(); docs.Add(doc); doc->Open();
MyApplication.CreateDocument():  return new MyDocument
```

  - Çizginin üstü framework, altı framework'ü kullanan uygulamadır. Open ve Close belge türüne bağlı olduğu için soyut; Save ve Revert (geri alma) türden bağımsız olduğu için somuttur.
  - Üst sınıf ortak adımları somut metotlarla yürütür, türe özgü işi (belgeyi oluşturmayı) alt sınıfa bırakır.
  - Generic sınıflar (Java 1.5 ile gelen generics, C++ class template) ile de çözülebilir, ama bu çözüm dile özeldir; kalıplar her nesneye dayalı dilde kullanılabilecek çözümler sunar.
- Kalıp yapısı (ekrandan, 00:17:20)
  - Soyut `Product` ve onun alt sınıfı `ConcreteProduct`; soyut `Creator` (`FactoryMethod()` soyut, `AnOperation()` somut) ve alt sınıfı `ConcreteCreator` (`FactoryMethod()` içinde `return new ConcreteProduct`). `AnOperation()` içinde `product = FactoryMethod()` çağrılır.
  - Hoca, `Creator`'dan `Product`'a da bir kesikli (kullanma) ok çizilebileceğini söyledi, çünkü `AnOperation()` içinde soyut ürün türü kullanılıyor.
  - Kalıp yalnız framework'ler için değildir; farklı türden ürünlerin oluşturulup kullanılması gereken her senaryoda kullanılabilir.

- Kalıbın uygulanabileceği anlar (ekrandan, 00:20:00)
  - Yeni nesneler oluşturması gereken bir sınıfın, oluşturacağı nesnenin türünü bilemediği yerlerde.
  - Bir sınıfın sorumluluğunu yardımcı alt sınıflara ileteceği ve hangi yardımcı alt sınıfın oluşturulacağı bilgisinin yerelleştirilmesi gerektiği yerlerde. Örnek: çizim uygulamasında şekillerin fareyle değiştirilmesi.
- Figure ve Manipulator örneği (ekrandan, 00:22:40)
  - `Client`, soyut `Figure` (`CreateManipulator()`) ve soyut `Manipulator` (`DownClick()`, `Drag()`, `UpClick()`) ile çalışır. `LineFigure` bir `LineManipulator`, `TextFigure` bir `TextManipulator` oluşturur.
  - Client bir figüre tıklayınca onun `CreateManipulator()` metodunu çağırır ve hangi somut manipülatörü aldığını bilmeden ortak işleri (tıklama, sürükleme, bırakma) yaptırır.
  - Özel düzenleyiciye ihtiyaç duymayan figürler ortak bir genel (generic) manipülatör kullanabilir.
- Factory Method uygulama örneği: e-ticaret sitesi (ekrandan, 00:30:00 ve 00:37:40)
  - Head First Design Patterns kitabından esinlenilmiştir. Farklı türde ürünler (elektronik, kitap vb.) satılır; site çalışanı siparişi işlerken adımlar aynı sırayla yürür ama her adımın iş mantığı ürün türüne göre değişir.
  - Soyut `Order` (`orderFromSupplier()`, `pack()`, `prepare()`, `sendToCustomer()`) Product; alt sınıfları `Electronics` ve `Books` Concrete Product. Soyut `Store` (`# createOrder(uniqueID : String) : Order`, `+ makeOrder()`) Creator; `ElectronicsStore` ve `BookStore` Concrete Creator.
  - Factory method `createOrder`'dır: `protected` ve soyut. `makeOrder` public metottur ve sırasıyla `prepare`, `orderFromSupplier`, `pack`, `sendToCustomer` çağırır.
  - Hoca derste parametre adını `type` yerine `uniqueID` olarak düzeltti, çünkü bu bir tür adı ya da enum değeri değil, kitap için ISBN/barkod, elektronik için ürün kodu gibi benzersiz bir tanımlayıcıdır.
- Java annotation'ları (ekrandan, 00:40:00)
  - `@` ile başlayan satırlar annotation'dır. `@SuppressWarnings("unused")` kullanılmayan üye alan uyarısını gizler. `@Override` bir metodun üst sınıftan gelen metodu yeniden tanımladığını (overriding) belirtir; Java'da konsa da konmasa da program aynı çalışır.
- Soru üzerine kalıtım ve çok biçimlilik gösterimi (01:00:00)
  - `Insan` sınıfı ve ondan türeyen `Calisan` (firma bilgili) ile `Ogrenci` (okul bilgili) sınıfları; her biri `toString()` metodunu yeniden tanımlar. `Insan` türünden bir dizide tutulsalar da her nesne kendini kendi alt sınıfının `toString()` metoduyla tanıtır. `Insan` soyut yapılınca da sonuç değişmez.
- Builder: amaç, örnek ve çözüm (ekrandan, 01:10:00)
  - Amaç: karmaşık bir nesnenin oluşturma işlemleriyle gösterim işlemlerini birbirinden ayırmak; böylece aynı oluşturma işlemi farklı gösterimler ortaya çıkarabilir.
  - Örnek: Rich Text Format (RTF) okuyucu. Belge düz ASCII metne, TeX biçimine ya da metni düzenlemeye yarayan bir GUI bileşenine (TextWidget) çevrilebilmeli; PDF gibi yeni biçimler sonradan kolayca eklenebilmeli.
  - `RTFReader.ParseRTF()` token token okur; `switch t.Type` ile `CHAR`, `FONT`, `PARA` için `builder->ConvertCharacter(t.Char)`, `builder->ConvertFontChange(t.Font)`, `builder->ConvertParagraph()` çağırır. Soyut `TextConverter`'ın alt sınıfları `ASCIIConverter`, `TeXConverter`, `TextWidgetConverter` sırasıyla `ASCIIText`, `TeXText`, `TextWidget` üretir.
  - Yeni biçim için case'lere dokunmadan yalnız yeni bir `TextConverter` alt sınıfı yazılır.
  - Çözümün eleştirileri: çok biçimlilik tam kullanılamadı, switch-case hâlâ duruyor (ama bu problemin doğasından gelir); alt sınıflar üst sınıfın tüm metotlarını gerçeklemek zorunda olduğu için anlamsız metotlar (örneğin ASCII için `ConvertFontChange`) boş gövdeli kalır.
- Builder kalıp yapısı (ekrandan, 01:16:00)

  - `Director` (`Construct()`) bir `Builder`'a (`BuildPart()` soyut) sahiptir; `ConcreteBuilder` (`BuildPart()`, `GetResult()`) `Product`'ı oluşturur. `Construct()`: `for all objects in structure { builder->BuildPart() }`.
  - Şemada switch-case görünmez ama `ConcreteBuilder.buildPart` metodunda kendini gösterir: parçalar parametre olarak aktarılmak zorundadır ve "parça şu ise şunu yap" gibi komutlar gerekir.
  - Client önce bir somut builder oluşturur, onu Director'a verir, `construct()` ile işi başlatır, sonunda sonucu builder'dan alır.

- Builder'ın uygulanabileceği anlar ve sonuçları (ekrandan, 01:20:00)
  - Karmaşık bir nesneyi oluşturma algoritmasının, nesneyi oluşturan parçalardan ve bu parçaların nasıl birleştirileceğinden bağımsız olması gerektiğinde.
  - Oluşturma sürecinin nesnenin farklı gösterimlerine izin vermesi gerektiğinde.
  - Sonuçlar: ürünün iç yapısı en az yan etkiyle değiştirilebilir (yeni bir ConcreteBuilder yazarak). Factory kalıplarına göre ürün oluşturma süreci üzerinde daha fazla denetim vardır: Director, Builder'a hangi işlemi hangi sırada yapacağını söyler.

- Builder uygulama örneği: araç bayisi (ekrandan, 01:30:00, 01:34:00, 01:40:00 ve 01:42:10)
  - Farklı marka ve modellerin satıldığı bir bayide, markalara özel adlandırma ifadelerine rağmen ortak bir araç modellemesi yapılacak. Araç; kasa, yakıt türü, motor hacmi ve gücü gibi bileşenlerin bir araya getirilmesiyle karmaşık bir süreçle oluşur.
  - `product` paketi: `Car` (`- brand, model : String`) → `Transmission` (`- gearCount : int {final}`, `- isAutomatic : boolean {final}`), `Engine` (`- volume, power : int`) → «enum» `FuelType {PETROLEUM, DIESEL, LPG, ELECTRIC}`, `Car` → «enum» `Chassis {SEDAN, HATCHBACK, STATIONWAGON, SUV}`. Düz oklar, şemada gösterilmese de ilgili türden üye alan olduğunu söyler.
  - Abstract Factory uygun değildir, çünkü bir marka "ailesi" sabit bileşenlerle gelmez; Factory Method ise bu karmaşıklık için basit kalır.
  - `CarBuilder` soyut sınıfı: `private Car product`, kurucuda `new Car(carModel)`, `getResult()` ürünün bir kopyasını döndürür, `protected getProduct()` alt sınıflar içindir, soyut `buildChassis()`, `buildEngine(String token)`, `buildTransmission(String token)`.
  - `HondaBuilder.buildEngine`: motor yoksa varsayılan `new Engine(FuelType.PETROLEUM, 1150, 60)`; token `ECO` içeriyorsa LPG, `PS` içeriyorsa beygir gücü, `L` içeriyorsa hacim ($1.6 \times 1000 = 1600$ cc).
  - `CarDirector` örneği: `new HondaBuilder("Civic Sedan")`, `{"Engine\t125PS", "Engine\t1.6L", "Gear\t6AT"}`; çıktı `Car: Honda Civic Sedan (Sedan)`, `Engine: Petroleum 1600cc 125HP`, `Transmission: 6 gears automatic transmission`.

- Prototype (ekrandan, 02:00:00, 02:05:10 ve 02:10:00)
  - Amaç: oluşturulacak nesnelerin türlerini belirlemek için prototip bir nesne kullanmak ve yeni nesneleri bu prototipi kopyalayarak oluşturmak.
  - Örnek: hazır bir grafik çerçeve programıyla müzik besteleme uygulaması. `Tool` (`Manipulate()`) altında `RotateTool` ve `GraphicTool`; `GraphicTool` bir `Graphic` prototipine sahiptir. `Graphic` (`Draw(Position)`, `Clone()`) altında `Staff` (porte) ve soyut `MusicalNote`; onun altında `WholeNote`, `HalfNote` (`Clone()`: `return copy of self`).
  - `GraphicTool.Manipulate()`: `p = prototype->Clone(); while (user drags mouse) { p->Draw(new position) } insert p into drawing`.
  - Abstract Factory ya da Factory Method ile her nota için ayrı fabrika sınıfı gerekirdi (ör. Do, Re, Mi… yedi fabrika); sınıf sayısı çok artar.
  - Yapı: `Client` (`Operation()`: `p = prototype->Clone()`) → soyut `Prototype` (`Clone()`) → `ConcretePrototype1`, `ConcretePrototype2`. Prototype nesnelerin kendini klonlaması için arayüz sunar; ConcretePrototype kullanılacak nesnelerdir; Client prototipin kendini kopyalamasını isteyerek yeni nesne oluşturur.
  - Uygulanabileceği anlar: sistem ürünlerinin nasıl oluşturulduğundan, birleştirildiğinden ve sunulduğundan bağımsız olmalıysa ve ek olarak şunlardan biri söz konusuysa (Factory Method'dan ayrışma): oluşturulacak sınıflar çalışma anında belirlenecekse; ürün hiyerarşisine paralel bir factory hiyerarşisi kurulmak istenmiyorsa; ürün sınıfının örneklerinin durum uzayı sınırlıysa (bu durumda sınıfın iyi tasarlanıp tasarlan-

madığı ayrıca tartışılır).
  - Kalıp C++ gibi sınıfların doğrudan işlenebilen varlıklar olmadığı dillerde daha yararlıdır; Smalltalk gibi geç bağlama (late binding) ya da Java gibi yansıtma (reflection) yetenekli dillerde dilin yetenekleri kullanılabilir, ama o çözüm dile özel olur.

- Sığ ve derin kopyalama (02:12:03)
  - Shallow copy yalnız nesnenin kendisini kopyalar; deep copy ilkel olmayan üye alanların da klonlarını oluşturur.
  - Derin kopyada döngüsel ilişkiler sorun çıkarır: anne, baba ve çocuklar ilişkisi olan bir `Insan` nesnesi derin kopyalanırken kendisinin birden fazla kopyası oluşabilir; döngüyü önlemek için `clone` metodu karmaşıklaşır.

- Prototype uygulama örneği: oyun için level editörü (ekrandan, 02:20:00 ve 02:23:30)
  - Kaynak Head First Design Patterns. Haritaya çok çeşitli canavarlar kolayca eklenmek isteniyor, ama canavarların çok sayıda durum bilgisi var: HP, DEX, AGL, LUCK, CHR vb.
  - «`Abstract`» `Monster` (Prototype): `# HP, DEX, AGL, LUCK, CHR : int`, `# type : String`, `+ clone() : Monster` (soyut), `+ getType() : String`, `# Monster(int, int, int, int, int)`. `GenericMonster` (ConcretePrototype): `+ clone()`, `+ GenericMonster(int ×5)`. `MonsterRegistry`: `+ findMonster(type : String) : Monster`, `+ registerMonster(Monster)`; canavarları tür adı anahtar olacak biçimde bir map içinde tutar. `MainApp` client'tır.
  - `GenericMonster.clone()` aynı durum bilgisine sahip yeni bir nesne döndürür; bütün üye alanlar ilkel olduğu için sığ/derin ayrımı gerekmez.
  - `main`: kayda `new GenericMonster(10, 10, 10, 10, 10)` eklenir; `reg.findMonster("Generic Monster").clone()` ile kopya alınır, `setHP(100)` ve `setType("Bolum sonu canavarı")` yapılıp kayda eklenir.

- Singleton (ekrandan, 02:40:00)
  - Amaç: bir sınıftan yalnız bir nesne oluşturulabilsin. Örnekler: yazdırma biriktiricisi (print spooler), çekirdek süreci, dosya sistemi yöneticisi, pencere yöneticisi.
  - İlk akla gelen statik bir üye alan tanımlamaktır; ama bu nesneyi hangi sınıfa koyacağımız sorusunu doğurur ve derleyicinin oluşturduğu tek kopya üzerinde söz hakkı bırakmaz.
  - Önerilen çözüm: sınıfa kendisinin tek bir örneğinin olmasını sağlama sorumluluğunu atamak. Sınıfın kendi türünden statik bir üyesi olur, kurucusu `private` tanımlanır, bu üye için bir factory metodu bulunur.
  - Şema: `Singleton` (`- uniqueInstance : Singleton {static}`, `- state : SingletonData`, `- Singleton(...)`, `+ getInstance() : Singleton {static}`, `+ singletonOperation()`, `+ getterMethods()`); `getInstance()`: `if (uniqueInstance == null) { uniqueInstance = new Singleton(...) } return uniqueInstance;`.
  - Singleton sınıfından kalıtımla başka singleton sınıfları türetilebilir. İleride görülecek Strategy ve State gerçeklemeleri iyi singleton adaylarıdır.

Hocanın Özellikle Vurguladığı Kısımlar

- Kalıpların birbirinden farkını iyi oturtun
  - Bir problemi en iyi hangi kalıpla çözebileceğinize ancak benzer kalıpların farklarını bilirseniz karar verebilirsiniz. Örnek soru tipi: savaş ve kargo uçakları üreten, uçakların farklı motor türleri gibi bileşenleri olan bir fabrika için Abstract Factory, Factory Method ve Builder'dan hangisi?
- Builder ile factory kalıplarının farkı
  - Builder'da Director, Builder'a hangi parçayı hangi sırayla ve nasıl oluşturacağını söyler; Factory Method ve Abstract Factory'de client yalnız "bana bir ürün ver" der, oluşturmanın ayrıntısına somut fabrika karar verir. Daha fazla denetim, daha fazla bağımlılık olarak da görülebilir.
- Kalıtımda "almıyorum" denemez
  - Alt sınıf üst sınıfın tüm üyelerini alır; değiştirebilir ya da yenisini ekleyebilir ama reddedemez. Hoca bu yüzden "inheritance"ı "miras" değil "kalıtım" diye çevirir: mi-

ras reddedilebilir, genetik özellikler reddedilemez, yalnız değiştirilebilir. Boş gövdeli metotlar bu yüzden bir kusurdur.
- Kesikli çizgi (kullanma ilişkisi)
  - Arada sahiplik yoksa, ama bir metodun içinde o sınıftan nesne oluşturuluyor/kullanılıyorsa ya da o sınıf bir metodun parametresiyse kesikli çizgi çizilir. Store ile Order arasındaki «Instantiate» çizgisi de bu yüzden kesikli olmalıdır.
- Protected metot
  - Factory method'un (`createOrder`) `protected` olması, onu alt sınıflara public, diğer sınıflara private gibi gösterir; client'ın keyfi çağırması engellenir.
- Tasarım kalıpları dilden bağımsızdır
  - Generic sınıflar ya da reflection gibi dile özel çözümler başka dillere taşınamaz; kalıplar her nesneye dayalı dilde uygulanabilir. Ders örnekleri Java'dır ama C#'a (package yerine namespace) ve C++'a kolayca çevrilebilir.
- Singleton ne zaman gerekli?
  - Tek bir kullanıcısı olacaksa statik üye yeter; farklı türden kullanıcılar aynı tek nesneyi kullanacaksa Singleton kalıbı kurulmalıdır.

Kısa Tekrar Notları

- Factory Method: nesnenin hangi sınıftan oluşturulacağına alt sınıflar karar verir. Bileşenler: Product, ConcreteProduct, Creator (soyut factory method + somut işlemler), ConcreteCreator.
- Factory Method örnekleri: Document/Application framework'ü, Figure/Manipulator, Store/Order (`createOrder` protected ve soyut).
- Builder: karmaşık nesnenin oluşturulmasını gösteriminden ayırır. Bileşenler: Director (`Construct()`), Builder (`BuildPart()`), ConcreteBuilder (`GetResult()`), Product.
- Builder'da Director süreci adım adım yönetir; yeni gösterim için yeni ConcreteBuilder yazılır.
- Builder eleştirisi: switch-case `buildPart`'ta kalır; alt sınıfta anlamsız metotlar boş gövdeli kalır.
- Prototype: yeni nesneler bir prototipin klonlanmasıyla oluşturulur. Bileşenler: Client, Prototype (`Clone()`), ConcretePrototype.
- Prototype; sınıflar çalışma anında belirleniyorsa, paralel factory hiyerarşisi istenmiyorsa, durum uzayı sınırlıysa seçilir.
- Shallow copy referansları kopyalar, deep copy alt nesneleri de klonlar; derin kopyada döngüsel ilişkilere dikkat.
- Singleton: statik `uniqueInstance`, `private` kurucu, statik `getInstance()`.
- Yaratımsal kalıplar tamamlandı: Abstract Factory, Factory Method, Builder, Prototype, Singleton.

Detaylı Açıklamalar

- Factory Method'da framework ve uygulama katmanı
  - Framework katmanında tanımlanan soyut metotlar, yeni uygulamaların kendine özgü işleri yapabilmesine fırsat verir. Her uygulamada aynı yapılan işler ise somut metotlarla, kimi zaman uygulama sınıfında kimi zaman belge sınıfında yürütülür. `NewDocument()` önce soyut `CreateDocument()` ile belgeyi oluşturtur (bunu somut alt sınıf yapar), sonra açık belgeler listesine ekler, sonra belgenin soyut `Open()` metodunu çağırır.
  - Revert (geri alma) gibi bir yetenek framework tarafından sağlanır; bunun için ileride görülecek Memento kalıbı kullanılabilir.
- E-ticaret örneğinin kodu (ekrandan, 00:37:40)

```
public abstract class Store {
    protected abstract Order createOrder( String uniqueID );
    public void makeOrder( String uniqueID ) {
        Order newOrder = createOrder( uniqueID );
        newOrder.prepare( );
        newOrder.orderFromSupplier( );
```

```
            newOrder.pack( );
            newOrder.sendToCustomer( );
        }
    }
    public class ElectronicsStore extends Store {
        protected Order createOrder( String uniqueID ) {
            Electronics newElectronicsOrder = new Electronics(uniqueID);
            //Elektronik malzeme siparişi için yapılacak işlemlerin tanımlanması
            return newElectronicsOrder;
        }
    }
    public class BookStore extends Store {
        protected Order createOrder(String uniqueID ) {
            Books newBookOrder = new Books(uniqueID);
            //Kitap siparişi için yapılacak işlemlerin tanımlanması
            return newBookOrder;
        }
    }
```

- `Order` tüm metotları soyut bir sınıftır; hocaya göre böyle sınıflar interface olarak da kodlanabilir, hatta hoca bunu tercih eder. Elektronik ürün için garanti belgesi koymak gibi türe özgü işler `Electronics` sınıfının gövdelerinde yer alır.
- Sınıf şemasında (ekrandan, 00:30:00) `Store` → `Order`, `ElectronicsStore` → `Electronics`, `BookStore` → `Books` «Instantiate» ilişkileri vardır. Şema bir Eclipse eklentisiyle koddan üretildiği için çizgiler düz çıkmıştır; hoca bunların kesikli olması gerektiğini söyledi.
- Ders sonrası not: Hoca `@Override` için "Java için bir anlam ifade etmez" dedi; program çalışırken fark yaratmadığı doğrudur, ama derleyici bu annotation'ı denetler ve metot gerçekten bir üst sınıf metodunu yeniden tanımlamıyorsa derleme hatası verir (büyük olasılıkla hocanın kastı çalışma zamanı davranışıdır).
- Ders sonrası not: Hoca C#'ta override etiketi konmazsa çok biçimliliğin çalışmadığını anlattı ve bunun değişip değişmediğinden emin olmadığını söyledi. C#'ta bu bir annotation değil `override` anahtar kelimesidir; üst sınıf metodunun da `virtual` (ya da `abstract`) olması gerekir. `override` yazılmazsa alt sınıf metodu üst sınıfınkini gizler ve üst sınıf türünden referansla çağrıldığında üst sınıfın metodu çalışır (büyük olasılıkla bu davranış güncel C# sürümlerinde de aynıdır).

- Builder ile araç örneğinin ayrıntıları (ekrandan, 01:40:00)
  - `getResult()` ürün oluşturulmamışsa `null`, oluşturulmuşsa `new Car(product.getModel())` ile yeni bir nesne kurup marka, şasi, motor ve vitesi kopyalayarak döndürür. Hoca ürünün kendisi yerine kopyasını döndürmeyi tercih ettiğini, bunun özel bir sebebi olmadığını söyledi.
  - `buildChassis()` parametre almaz, çünkü şasi genelde model adından anlaşılır. Honda'da model adında "back" geçiyorsa hatchback, "CRV" geçiyorsa SUV, ikisi de yoksa sedan. Renault'da Captur SUV'dur; diğer model adları için altyazı belirsizdir.
  - `HondaBuilder.buildEngine(String token)` (ekrandan, 01:40:00):

```
    public void buildEngine(String token) {
        //Min. values for a Honda engine
        if( getProduct().getEngine() == null )
            getProduct().setEngine( new Engine(FuelType.PETROLEUM, 1150, 60) );
        token = token.toUpperCase();
        if( token.contains("ECO") )
            getProduct().getEngine().setFuel(FuelType.LPG);
        else if( token.contains("PS") ) {
            int end = token.indexOf("PS");
            token = token.substring(0, end);
            getProduct().getEngine().setPower( Integer.parseInt(token) );
        }
        else if( token.contains("L") ) {
            int end = token.indexOf("L");
```

```
            token = token.substring(0, end);
            getProduct().getEngine().setVolume( (int)(Double.parseDouble(token) * 1000) );
        }
    }
```

  - Vites: token "AT" içeriyorsa otomatik; vites sayısı verilmemişse otomatikte 4, düzde 5 vites varsayılır. Önceki örnekteki switch-case burada if-else if zinciridir; çalışma mantığı aynıdır.
  - Çalıştırma (ekrandan, 01:42:10): `125PS` → 125 beygir, `1.6L` → 1.6 × 1000 = 1600 cc, `6AT` → 6 ileri otomatik. Motor türü belirtilmediği için varsayılan benzinli (Petroleum) kalır.

- Prototype ile factory kalıplarının farkı
  - Prototype'ta prototipin ilk nasıl oluşturulduğu client için şeffaftır; client yalnız prototipi alır, kopyalar ve birkaç niteliğini değiştirerek yeni varlığı elde eder.
  - Oyun karakteri örneği: savaşçı ve büyücü sınıflarının bazı niteliklerini içeren suikastçı gibi yeni bir sınıf sonradan, çalışma anında belirlenebilir.

- Sığ kopya, derin kopya ve döngüsellik
  - `Insan` sınıfında anne ve baba (birer `Insan`) ile çocuklar (`Insan` dizisi, `0..*`) olsun. Bir insanı derin kopyalarken annesinin ve babasının klonları oluşturulur; onların çocukları arasında yine aynı insan bulunduğu için onun da klonu oluşturulur. Sonuçta ilk klon ile annenin ve babanın çocuk listesindeki klonlar aynı nesne olmayabilir; bir kopya istenirken üç kopya oluşur.
  - Ders sonrası not: Hoca sığ kopyada ilkel olmayan üye alanların "null olarak kopyalandığını" söyledi. Doğrusu: sığ kopyada bu alanların referansları kopyalanır; kopya ile asıl nesne aynı alt nesneleri paylaşır (Java'da `Object.clone()` varsayılan olarak böyle çalışır). Hocanın "shallow copy yaparsak ebeveynlik ilişkileri bozulur" sözü de büyük olasılıkla bu paylaşımın yol açtığı tutarsızlık anlamında okunmalıdır.
  - Şemalarda gösterilmese de klonların üye alanlarına değer atamak ve okumak için getter/setter metotları gerekir.

- Prototype'ın üçüncü koşulu (durum uzayı) üzerine tartışma (02:25:00)
  - Slayt: "Ürün sınıfının örneklerinin durum uzayı sınırlı ise; bu durumda sınıfın iyi tasarlanıp tasarlanmadığı ayrıca tartışılır" (ekrandan, 02:10:00). Hoca anlatırken bu maddeyle ne kastettiğini tam hatırlamadığını söyledi.
  - Araç örneği sorusunda hoca arabada durum uzayının sınırlı, canavar örneğinde ise geniş olduğunu, bu yüzden Prototype'ın araba için uygun olmadığını söyledi; ardından bunun slayttaki maddenin tersi olduğunu fark etti ve slayttaki ölçütün Prototype'ı Builder ile değil factory kalıplarıyla karşılaştırdığını belirtti.
  - Ders sonrası not: GoF kitabında bu koşul büyük olasılıkla "bir sınıfın örnekleri yalnız birkaç farklı durum birleşiminden birine sahip olabiliyorsa, her birleşim için bir prototip kurup klonlamak, sınıfı her seferinde uygun durumla elle oluşturmaktan daha kolaydır" biçimindedir; yani slayttaki "sınırlı" ifadesi doğrudur.

- Singleton ile statik üye alanın farkı
  - Statik üye alan yalnız tanımlandığı sınıf içinde ortak kullanım sağlar ve derleyicinin oluşturduğu tek kopyaya müdahale şansı az kalır. Singleton'da nesne kendi sınıfı tarafından yönetilir; operasyon metotlarıyla davranışı üzerinde daha fazla söz hakkı olur. Sorumluluğun ilgili sınıfa verilmesi uyumu (cohesion) da yükseltir.
  - Statik alan başka bir sınıfa konsaydı, oradan kalıtımla yeni singleton'lar türetmek mümkün olmazdı; Singleton başlı başına bir sınıf olduğu için kalıtımla yeni singleton sınıfları kolayca türetilir.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.